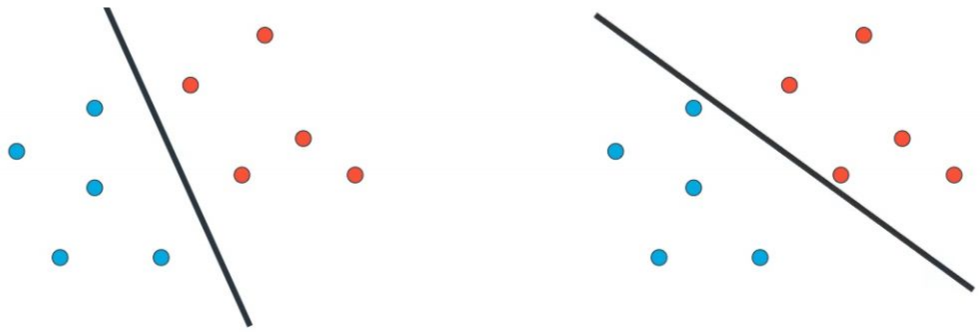
Support Vector Machine

Kritanta Saha
Assistant Professor
Dept. of Computer Science & Engineering
Sister Nivedita University

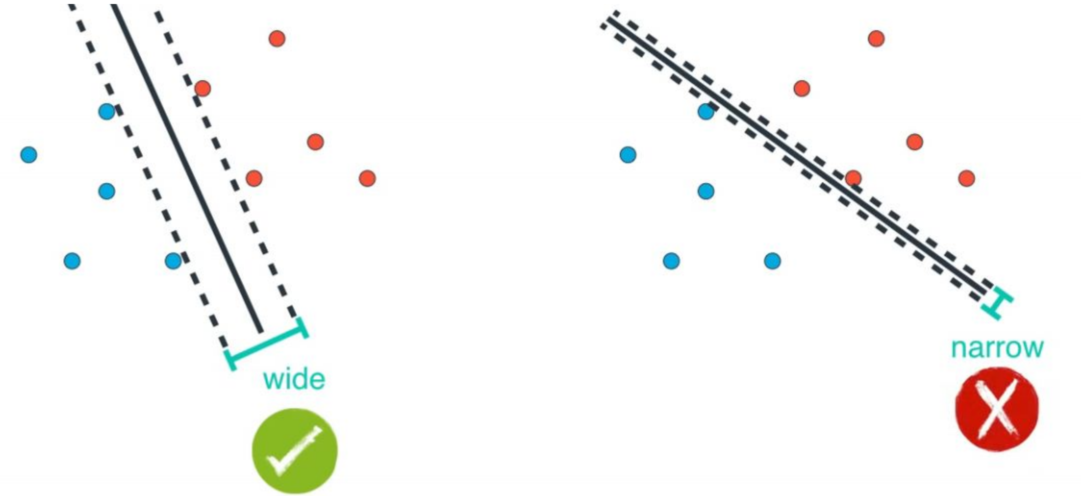
For Classification problem

Which line is better?

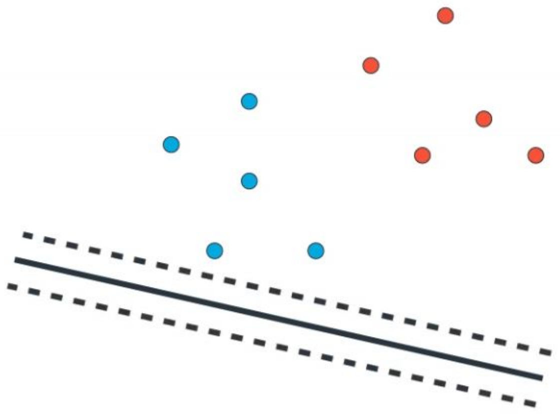


For Classification problem

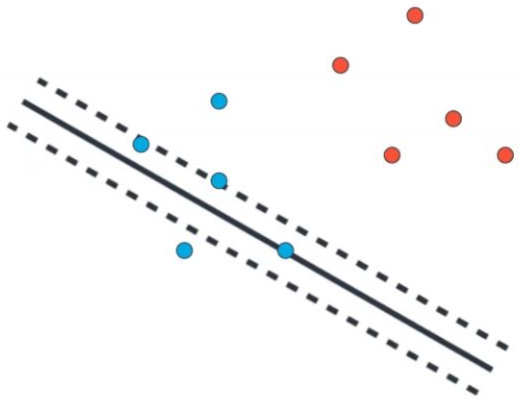
Which line is better?



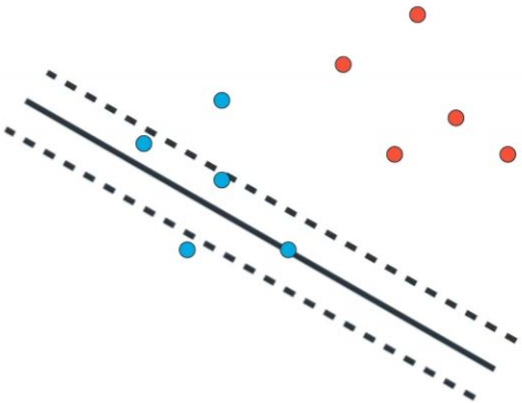
How to split data while separating line



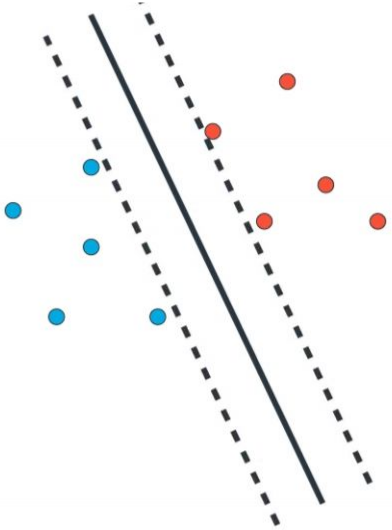
How to split data while separating line



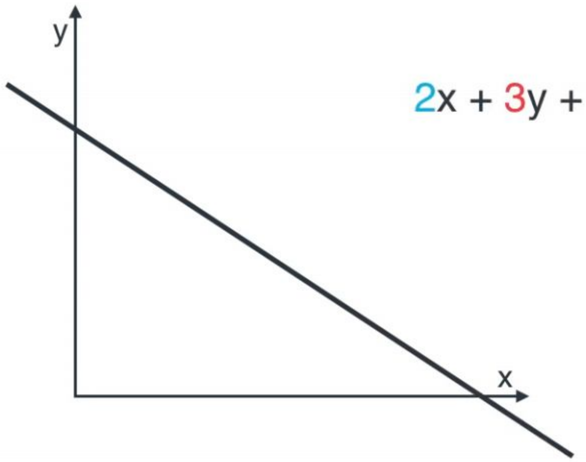
How to split data while separating line



How to split data while separating line

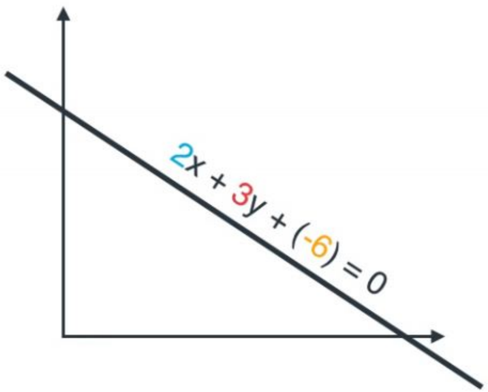


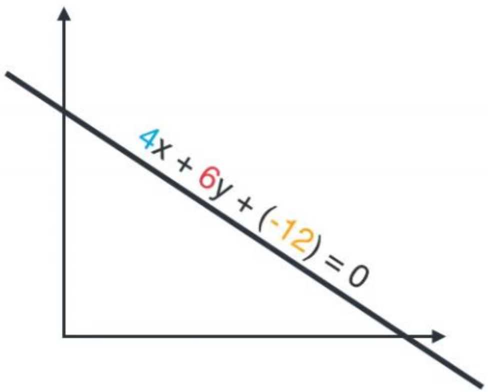
How to separate lines



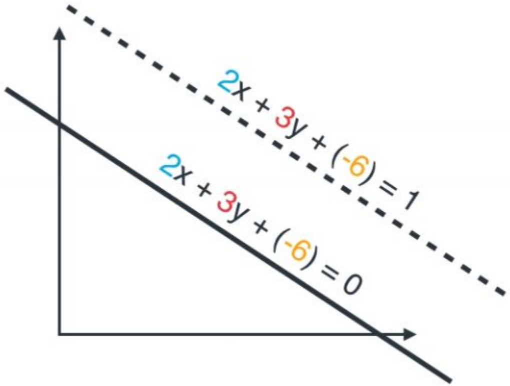
$$2x + 3y + (-6) = 0$$

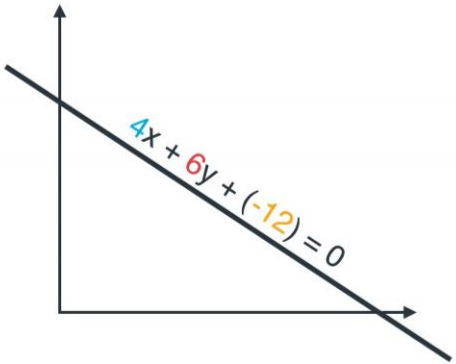
How to separate lines



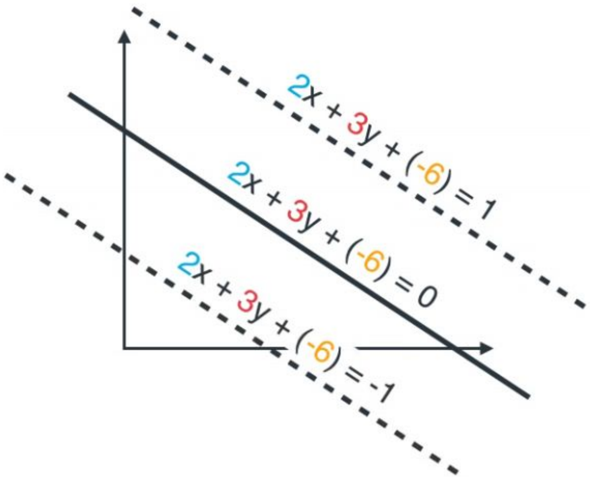


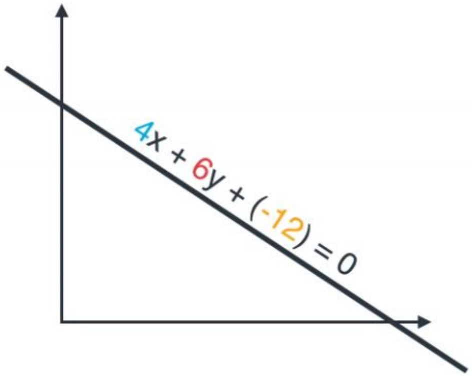
How to separate lines



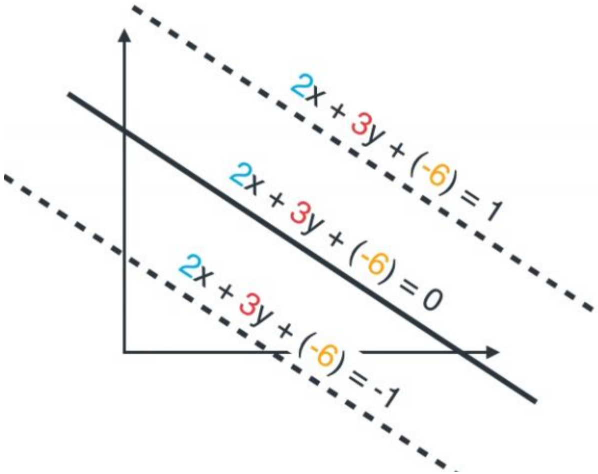


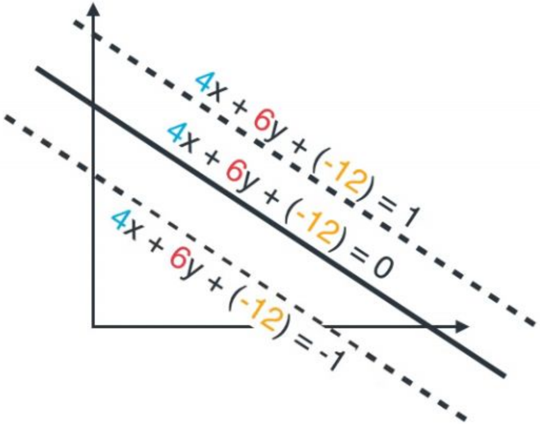
How to separate lines



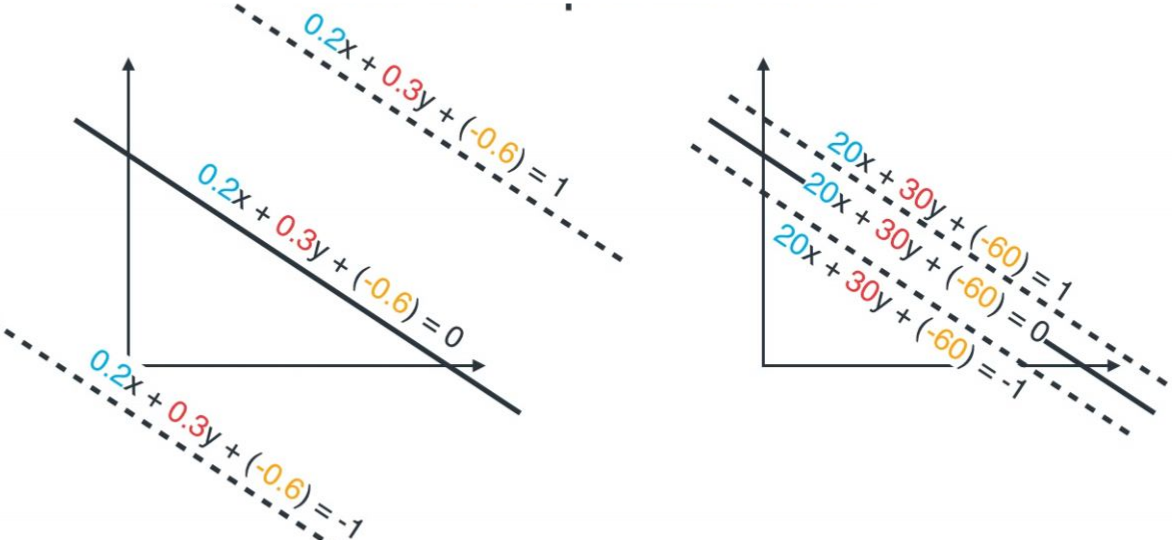


How to separate lines

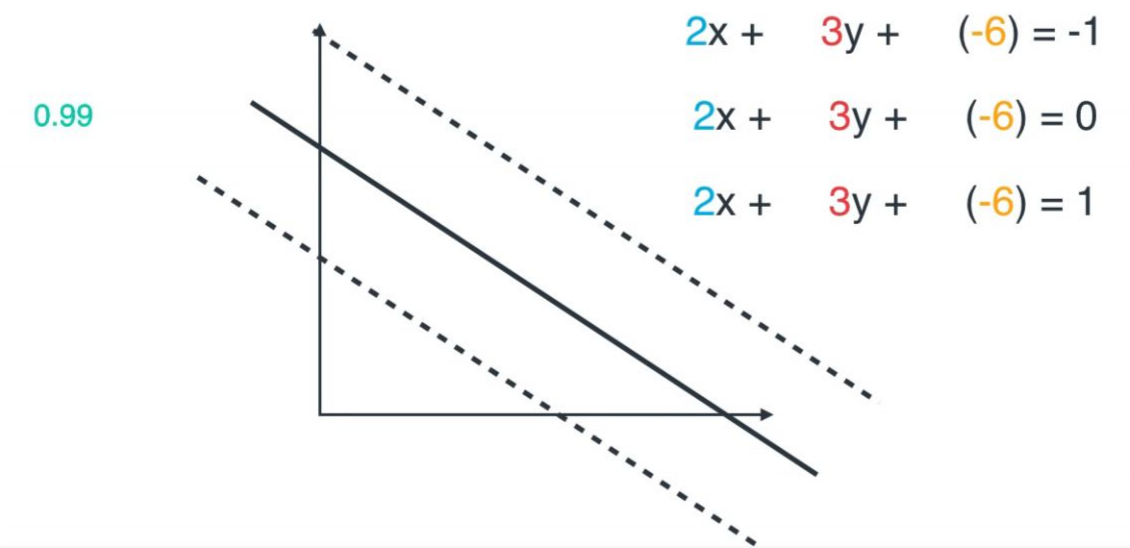




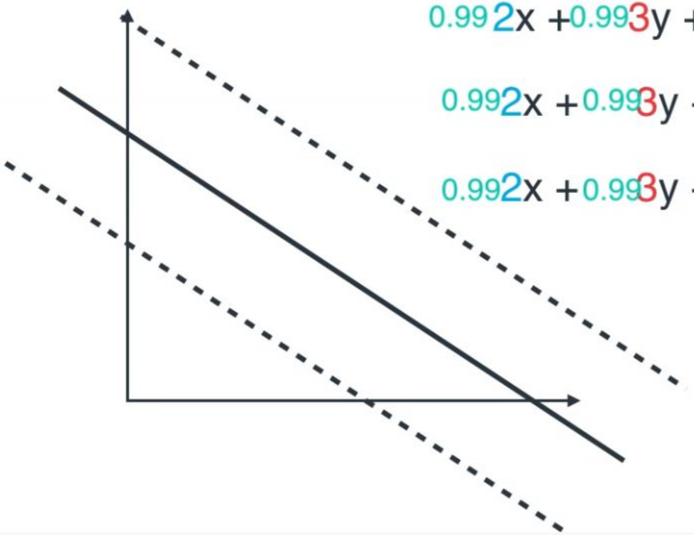
How to separate lines



Expanding rate



Expanding rate

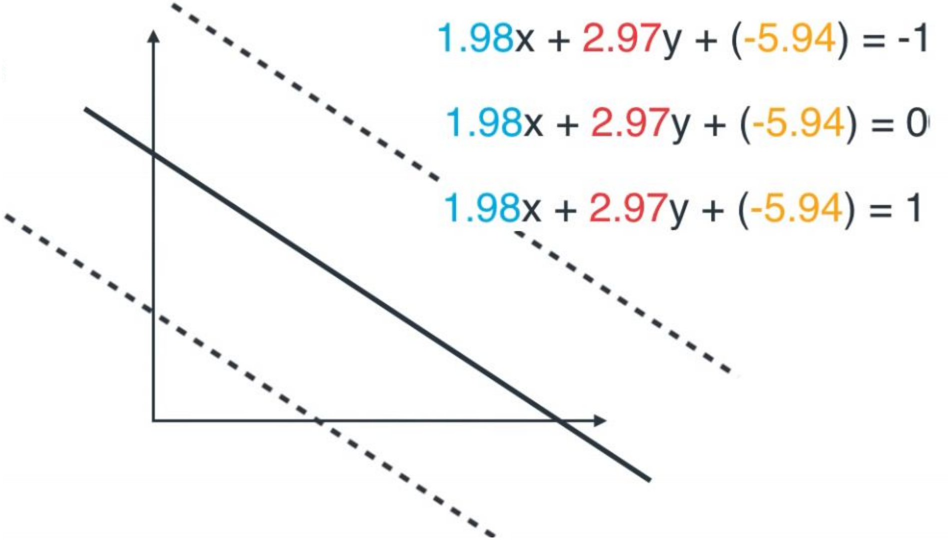


$$0.99\,2x + 0.99\,3y + 0.99(-6) = -1$$

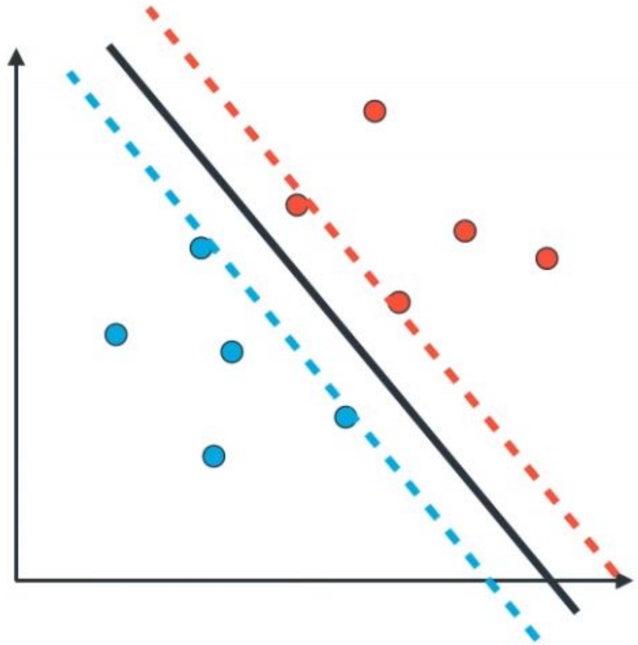
$$0.99\,2x + 0.99\,3y + 0.99(-6) = 0$$

$$0.99\,2x + 0.99\,3y + 0.99(-6) = 1$$

Expanding rate



SVM algorithm



Step 1: Start with a line, and two equidistant parallel lines to it.

Step 2: Pick a large number. 1000 (number of repetitions, or epochs)

Step 3: Pick a number close to 1. (the expanding factor) 0.99

Step 4: (repeat 1000 times)

- Pick random point
- If point is correctly classified:
  - Do nothing
- If point is incorrectly classified
  - Move line towards point
- Separate the lines using the expanding factor

Step 5: Enjoy your lines that separate the data!

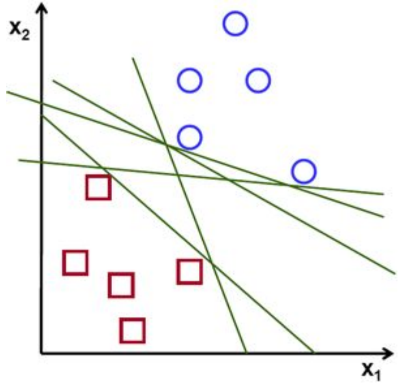
Definition

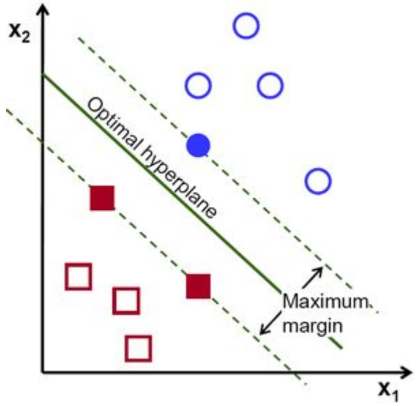
- A **Support Vector Machine (SVM)** is a classifier defined by a separating hyperplane.

- In other words, given labeled training data (*supervised learning*), the algorithm outputs an **optimal hyperplane** which categorizes new examples.

- In two dimensional space this hyperplane is a **line** dividing a plane in two parts where in each class lay in either side.

- Our objective is to find a plane that has the **maximum margin, i.e the maximum distance between data points of both classes.**





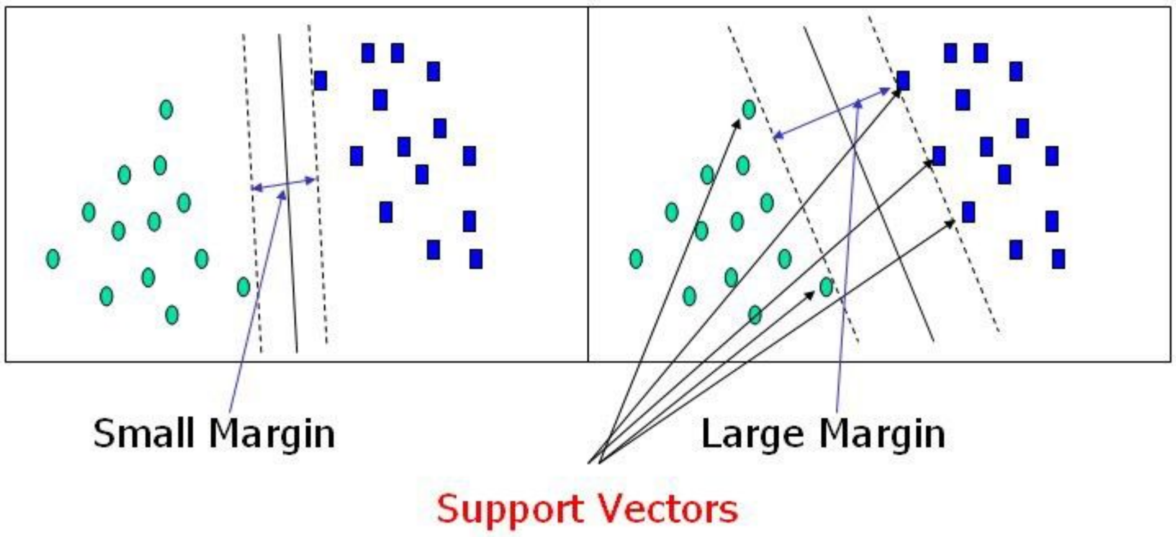
SVM: properties

- Data points falling on either side of the hyperplane can be attributed to different **classes**.
- Also, the dimension of the hyperplane depends upon the **number of features.**
- If the number of input features is 2, then the hyperplane is just a line.
- If the number of input features is 3, then the hyperplane becomes a two-dimensional plane.

Support Vectors

- **Support vectors** are data points that are **closer to the hyperplane** and influence the position and orientation of the hyperplane.

- Using these support vectors, we maximize the margin of the classifier.



Large margin intuition

- In SVM, we take the output of the linear function and if that output is greater than 1, we identify it with one class and if the output is less than -1, we identify is with another class.
- Since the threshold values are changed to 1 and -1 in SVM, we obtain this reinforcement range of values([-1,1]) which acts as margin.

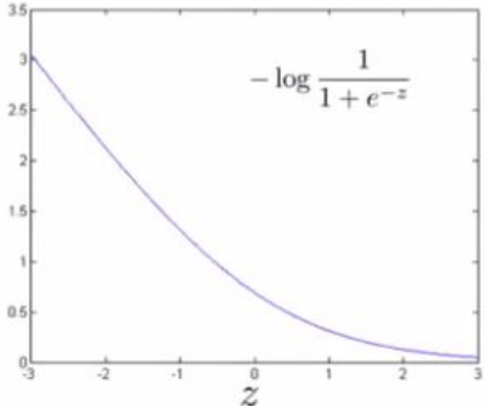
Previous Cost Function for logistic regression

For each example of Logistic regression

Cost of example: $-(y \log h_\theta(x) + (1-y)\log(1-h_\theta(x)))$

$$= -y \log \frac{1}{1+e^{-\theta^T x}} - (1-y)\log(1 - \frac{1}{1+e^{-\theta^T x}})$$

If $y = 1$ (want $\theta^T x \gg 0$):



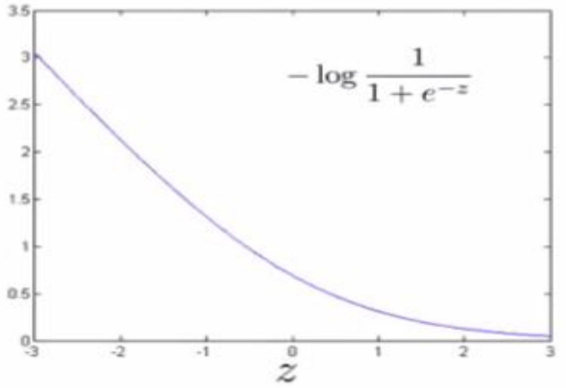
Modified Cost Function for SVM

For each example of Logistic regression

Cost of example: $-(y \log h_\theta(x) + (1-y)\log(1-h_\theta(x)))$

$$= -y \log \frac{1}{1+e^{-\theta^T x}} - (1-y)\log(1 - \frac{1}{1+e^{-\theta^T x}})$$

If $y = 1$ (want $\theta^T x \gg 0$):



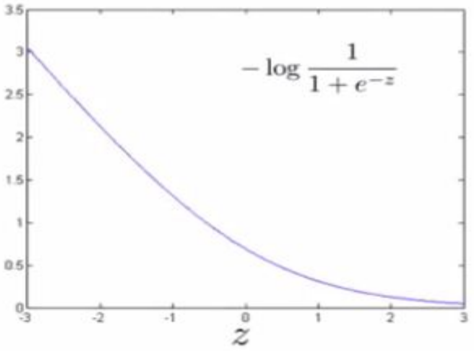
Modified Cost Function for SVM

For each example of Logistic regression

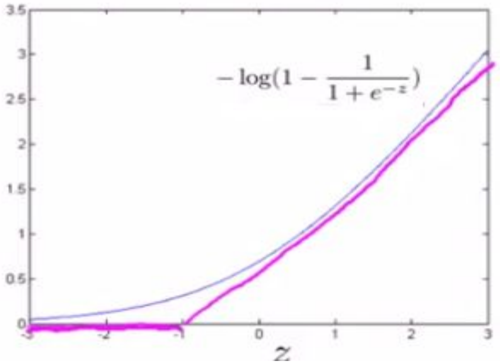
Cost of example: $-(y\log h_\theta(x) + (1-y)\log(1-h_\theta(x)))$

$$= -y\log\frac{1}{1+e^{-\theta^T x}} - (1-y)\log(1-\frac{1}{1+e^{-\theta^T x}})$$

If $y=1$ (want $\theta^T x \gg 0$):

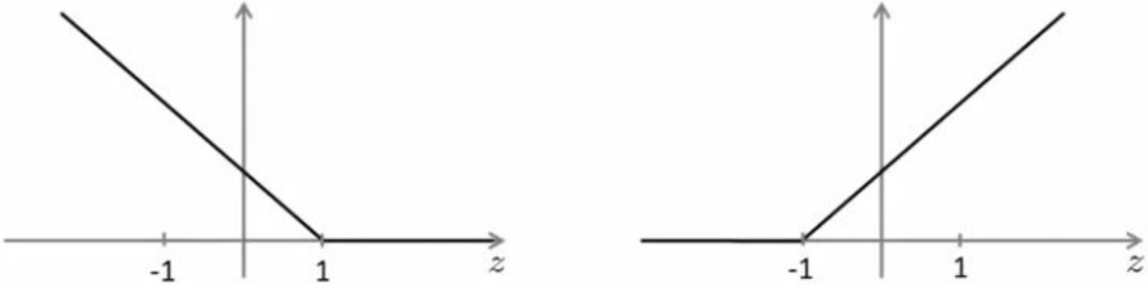


If $y=0$ (want $\theta^T x \ll 0$):



Cost Function

$$\min_{\theta} C\sum_{i=1}^{m}\left[y^{(i)}cost_1(\theta^T x^{(i)}) + (1-y^{(i)})cost_0(\theta^T x^{(i)})\right] + \frac{1}{2}\sum_{i=1}^{n}\theta_j^2$$



If $y = 1$, we want $\theta^T x \geq 1$ (not just ≥ 0)

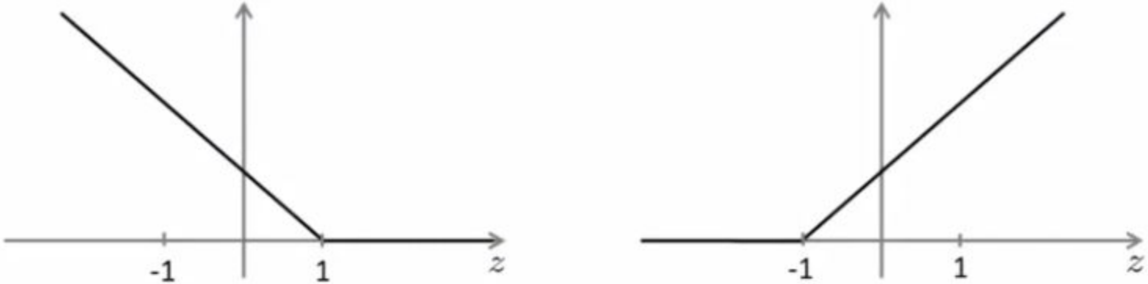
If $y = 0$, we want $\theta^T x \leq -1$ (not just < 0)

SVM Hypothesis:

$$h_\theta(x) = \begin{cases} 1 & \text{if } \theta^T x >= 0 \\ 0 & \text{otherwise} \end{cases}$$

Cost Function

$$\min_\theta C \sum_{i=1}^{m} \left[y^{(i)} cost_1(\theta^T x^{(i)}) + (1 - y^{(i)}) cost_0(\theta^T x^{(i)}) \right] + \frac{1}{2} \sum_{i=1}^{n} \theta_j^2$$



If $y = 1$, we want $\theta^T x \geq 1$ (not just ≥ 0)

If $y = 0$, we want $\theta^T x \leq -1$ (not just < 0)

SVM Hypothesis:

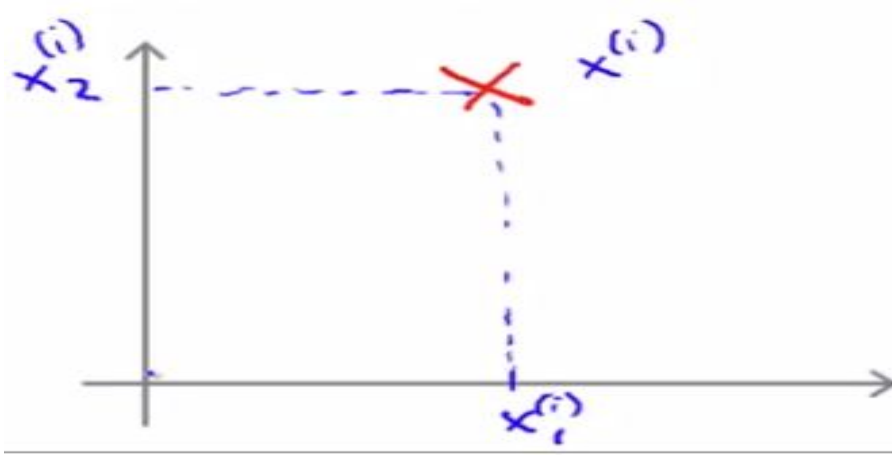
$$h_\theta(x) = \begin{cases} 1 & \text{if } \theta^T x >= 0 \\ 0 & \text{otherwise} \end{cases}$$

Now, if C is very large then minimizing this optimization objective will make first term equal to zero.

SVM decision boundary

Objective becomes minimizing the norm of the square length of the parameter vector theta.

$$\min_{\theta} \frac{1}{2} \sum_{j=1}^{n} \theta_j^2 = \frac{1}{2} \|\theta\|^2$$



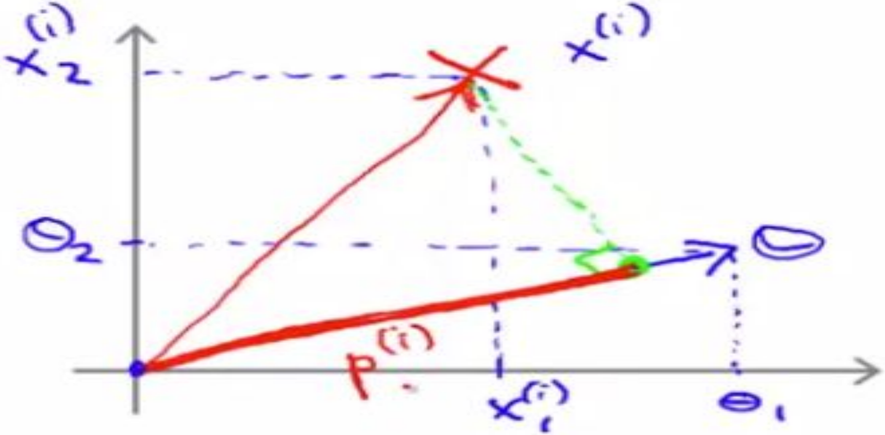
SVM decision boundary

Objective becomes minimizing the norm of the square length of the parameter vector theta.

$$\min_{\theta} \frac{1}{2}\sum_{j=1}^{n}\theta_j^2 = \frac{1}{2}\|\theta\|^2$$

$$\theta^T x^{(i)} = p^{(i)}\,\|\theta\|$$

$$= \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)}$$



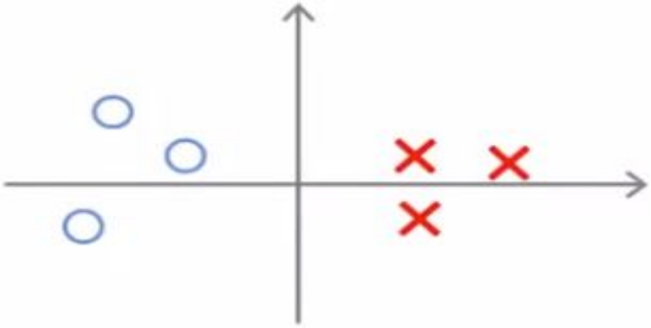
Why SVM will maximize margin?

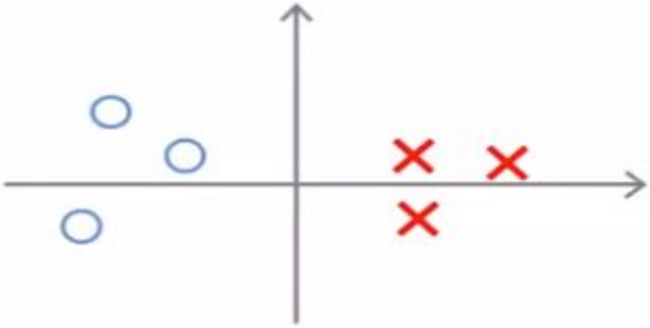
$$\min_{\theta} \frac{1}{2}\sum_{j=1}^{n}\theta_j^2 = \frac{1}{2}\|\theta\|^2$$

$$\begin{aligned} \text{s.t.} \quad & p^{(i)} \cdot \|\theta\| \geq 1 \quad && \text{if } y^{(i)} = 1 \\ & p^{(i)} \cdot \|\theta\| \leq -1 \quad && \text{if } y^{(i)} = 0 \end{aligned}$$

where $p^{(i)}$ is the projection of $x^{(i)}$ onto the vector θ.

Simplification: $\theta_0 = 0$





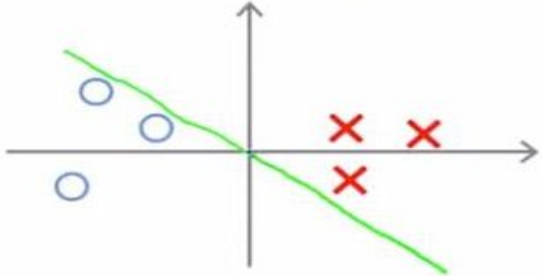
Why SVM will maximize margin?

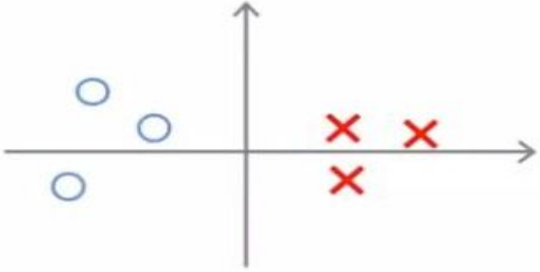
$$\min_{\theta} \frac{1}{2}\sum_{j=1}^{n}\theta_j^2 = \frac{1}{2}\|\theta\|^2$$

$$\text{s.t.} \quad p^{(i)} \cdot \|\theta\| \geq 1 \qquad \text{if } y^{(i)} = 1$$
$$p^{(i)} \cdot \|\theta\| \leq -1 \qquad \text{if } y^{(i)} = 0$$

where $p^{(i)}$ is the projection of $x^{(i)}$ onto the vector θ.

Simplification: $\theta_0 = 0$





Why SVM will maximize margin?

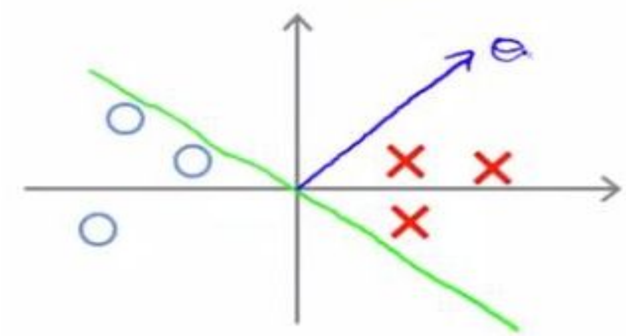
$$\min_{\theta} \frac{1}{2} \sum_{j=1}^{n} \theta_j^2 = \frac{1}{2} \|\theta\|^2$$

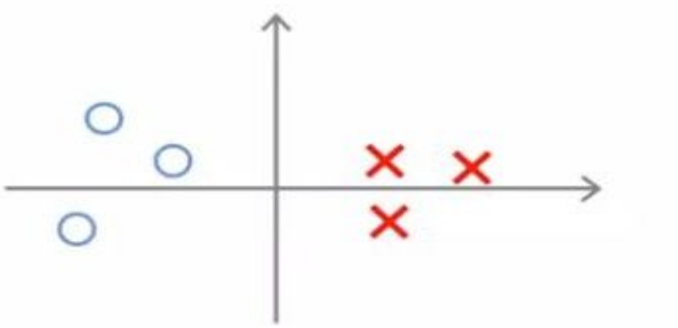
$$\text{s.t.} \quad p^{(i)} \cdot \|\theta\| \geq 1 \qquad \text{if } y^{(i)} = 1$$

$$\qquad p^{(i)} \cdot \|\theta\| \leq -1 \qquad \text{if } y^{(i)} = 0$$

where $p^{(i)}$ is the projection of $x^{(i)}$ onto the vector θ.

Simplification: $\theta_0 = 0$





Why SVM will maximize margin?

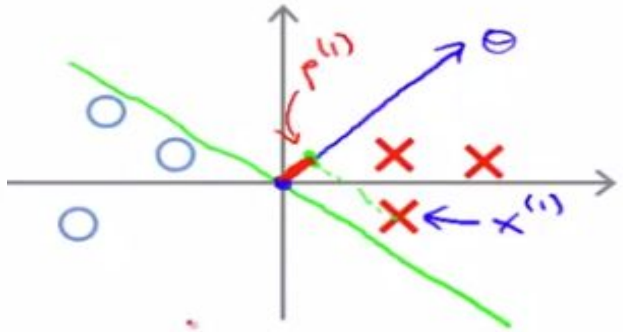
$$\min_{\theta} \frac{1}{2} \sum_{j=1}^{n} \theta_j^2 = \frac{1}{2} \|\theta\|^2$$

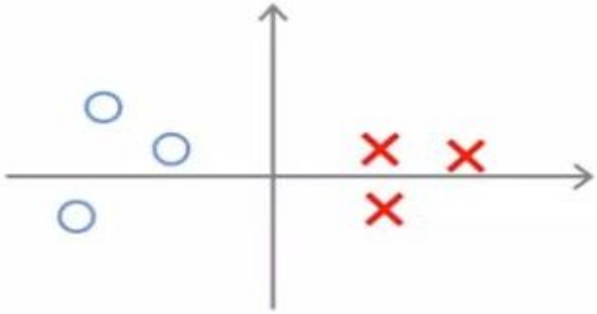
$$\text{s.t.} \quad p^{(i)} \cdot \|\theta\| \geq 1 \quad \text{if } y^{(i)} = 1$$

$$p^{(i)} \cdot \|\theta\| \leq -1 \quad \text{if } y^{(i)} = 0$$

where $p^{(i)}$ is the projection of $x^{(i)}$ onto the vector θ.

Simplification: $\theta_0 = 0$





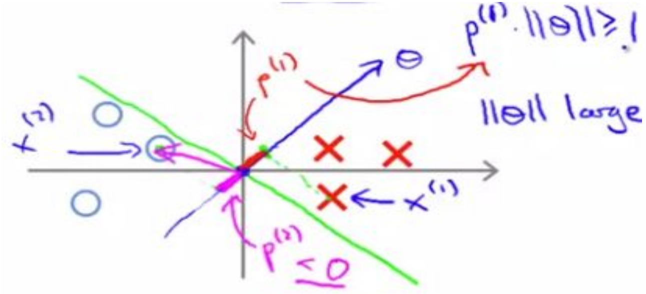
Why SVM will maximize margin?

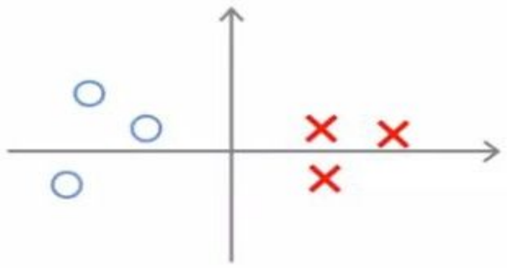
$$\min_{\theta} \frac{1}{2}\sum_{j=1}^{n}\theta_j^2 = \frac{1}{2}\|\theta\|^2$$

s.t. $p^{(i)} \cdot \|\theta\| \geq 1 \quad \text{if } y^{(i)} = 1$

$p^{(i)} \cdot \|\theta\| \leq -1 \quad \text{if } y^{(i)} = 0$

where $p^{(i)}$ is the projection of $x^{(i)}$ onto the vector θ.
Simplification: $\theta_0 = 0$





But θ should be small

Why SVM will maximize margin?

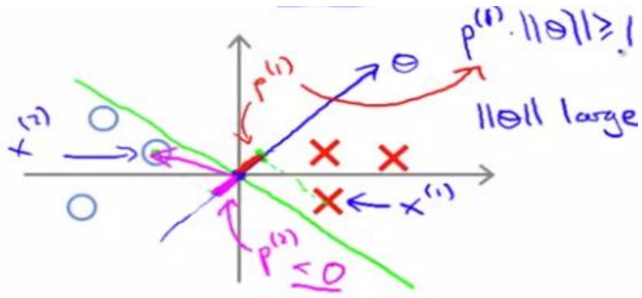
$$\min_{\theta} \frac{1}{2}\sum_{j=1}^{n}\theta_j^2 = \frac{1}{2}\|\theta\|^2$$

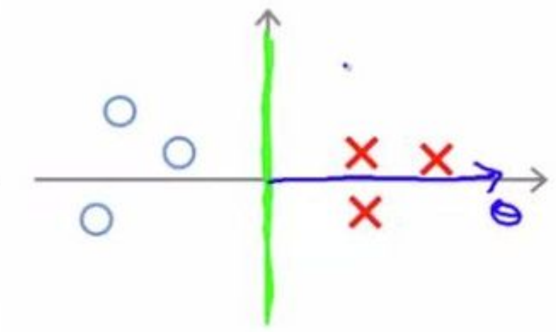
s.t. $p^{(i)} \cdot \|\theta\| \geq 1 \quad$ if $y^{(i)} = 1$

$p^{(i)} \cdot \|\theta\| \leq -1 \quad$ if $y^{(i)} = 0$

where $p^{(i)}$ is the projection of $x^{(i)}$ onto the vector θ.

Simplification: $\theta_0 = 0$





But θ should be small

Why SVM will maximize margin?

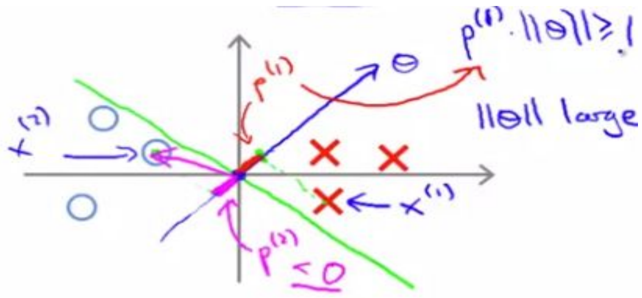
$$\min_{\theta} \frac{1}{2} \sum_{j=1}^{n} \theta_j^2 = \frac{1}{2} \|\theta\|^2$$

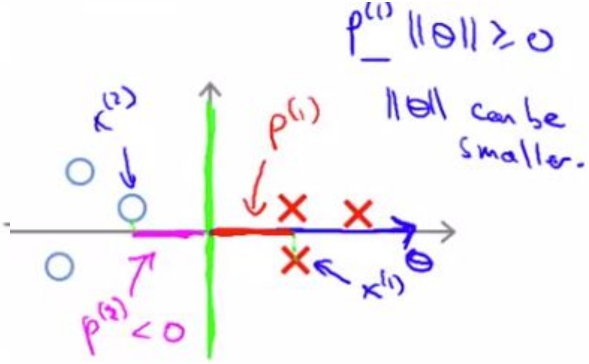
$$\text{s.t.} \quad p^{(i)} \cdot \|\theta\| \geq 1 \quad \text{if } y^{(i)} = 1$$

$$p^{(i)} \cdot \|\theta\| \leq -1 \quad \text{if } y^{(i)} = 0$$

where $p^{(i)}$ is the projection of $x^{(i)}$ onto the vector θ.

Simplification: $\theta_0 = 0$





But θ should be small, so summation of $p^{(i)}$ should be large, i.e. the margin.

Lagrange duality

- Consider a constrained optimization problem:

$$\begin{aligned} \min_w \quad & f(w) \\ \text{s.t.} \quad & h_i(w) = 0, \quad i = 1, \ldots, l. \end{aligned}$$

- In this method, we define the Lagrangian to be:

$$\mathcal{L}(w, \beta) = f(w) + \sum_{i=1}^{l} \beta_i h_i(w)$$

- Here, the βi ’s are called the Lagrange multipliers. We would then find and set L’s partial derivatives to zero: and solve for w and β.

$$\frac{\partial \mathcal{L}}{\partial w_i} = 0; \quad \frac{\partial \mathcal{L}}{\partial \beta_i} = 0,$$

- We will generalize this to constrained optimization problems in which we may have inequality as well as equality constraints.

Lagrange duality

- Consider the following, which we'll call the primal optimization problem:
- To solve it, we start by defining the generalized Lagrangian Here, the αi 's and βi 's are the Lagrange multipliers.
- Consider the quantity: Here, the "P" subscript stands for "primal."

$$\begin{aligned} \min_w \quad & f(w) \\ \text{s.t.} \quad & g_i(w) \le 0, \quad i = 1, \ldots, k \\ & h_i(w) = 0, \quad i = 1, \ldots, l. \end{aligned}$$

$$\mathcal{L}(w, \alpha, \beta) = f(w) + \sum_{i=1}^{k} \alpha_i g_i(w) + \sum_{i=1}^{l} \beta_i h_i(w).$$

$$\theta_{\mathcal{P}}(w) = \max_{\alpha, \beta \,:\, \alpha_i \ge 0} \mathcal{L}(w, \alpha, \beta).$$

$$\begin{aligned} \theta_{\mathcal{P}}(w) &= \max_{\alpha, \beta \,:\, \alpha_i \ge 0} f(w) + \sum_{i=1}^{k} \alpha_i g_i(w) + \sum_{i=1}^{l} \beta_i h_i(w) \\ &= \infty. \end{aligned}$$

Lagrange duality

- Conversely, if the constraints are indeed satisfied for a particular value of w, then $\theta_P(w) = f(w)$. Hence:
- Thus, θ_P takes the same value as the objective in our problem for all values of w that satisfies the primal constraints, and is positive infinity if the constraints are violated.
- Hence, if we consider the minimization problem we see that it is the same problem
- We also define the optimal value of the objective to be $p^* = \min_w \theta_P(w)$; we call this the **value** of the primal problem.

$$\theta_{\mathcal{P}}(w) = \begin{cases} f(w) & \text{if } w \text{ satisfies primal constraints} \\ \infty & \text{otherwise.} \end{cases}$$

$$\min_w \theta_{\mathcal{P}}(w) = \min_w \max_{\alpha,\beta\,:\,\alpha_i \geq 0} \mathcal{L}(w, \alpha, \beta),$$

Lagrange duality

- Now, let's look at a slightly different problem. We define:
- Here, the "D" subscript stands for "dual." Note also that whereas in the definition of θ_P we were optimizing (maximizing) with respect to α, β, here we are minimizing with respect to w.
- We can now pose the dual optimization problem
- We also define the optimal value of the dual problem's objective to be

$$\min_w \theta_{\mathcal{P}}(w) = \min_w \max_{\alpha,\beta\,:\,\alpha_i \geq 0} \mathcal{L}(w,\alpha,\beta),$$

$$\theta_{\mathcal{D}}(\alpha,\beta) = \min_w \mathcal{L}(w,\alpha,\beta).$$

$$\max_{\alpha,\beta\,:\,\alpha_i \geq 0} \theta_{\mathcal{D}}(\alpha,\beta) = \max_{\alpha,\beta\,:\,\alpha_i \geq 0} \min_w \mathcal{L}(w,\alpha,\beta).$$

$$d^* = \max_{\alpha,\beta\,:\,\alpha_i \geq 0} \theta_{\mathcal{D}}(w).$$

KKT Conditions

However, under certain conditions, we will have $d^* = p^*$, so that we can solve the dual problem in lieu of the primal problem.

We assume there are w^*, α^*, β^* so that w^* is the solution to the primal problem, α^*, β^* are the solution to the dual problem, and moreover $p^* = d^* = L(w^*, \alpha^*, \beta^*)$. Moreover, w^*, α^*, β^* satisfy the Karush-Kuhn-Tucker (KKT) conditions, which are as follows, then it is also a solution to the primal and dual problems.

$$
\begin{aligned}
\frac{\partial}{\partial w_i}\mathcal{L}(w^*, \alpha^*, \beta^*) &= 0, \quad i = 1, \ldots, n \\
\frac{\partial}{\partial \beta_i}\mathcal{L}(w^*, \alpha^*, \beta^*) &= 0, \quad i = 1, \ldots, l \\
\alpha_i^* g_i(w^*) &= 0, \quad i = 1, \ldots, k \\
g_i(w^*) &\leq 0, \quad i = 1, \ldots, k \\
\alpha^* &\geq 0, \quad i = 1, \ldots, k
\end{aligned}
$$

=> KKT dual complementarity condition.

KKT Conditions

- The KKT dual complementarity condition, specifically, it implies that if $\alpha^*_i > 0$, then $g_i(w^*) = 0$. (I.e., the “$g_i(w) \leq 0$” constraint is active, meaning it holds with equality rather than with inequality.)
- Later on, this will be key for showing that the SVM has only a small number of “support vectors”
- The KKT dual complementarity condition will also give us our convergence test when we talk about the SMO algorithm.

Optimal margin classifiers

- The (primal) optimization problem for finding the optimal margin classifier:

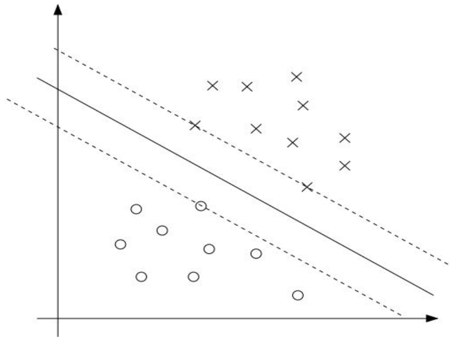
$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2}||w||^2 \\ \text{s.t.} \quad & y^{(i)}(w^T x^{(i)} + b) \geq 1, \quad i = 1, \ldots, m \end{aligned}$$

- We can write the constraints as

$$g_i(w) = -y^{(i)}(w^T x^{(i)} + b) + 1 \leq 0.$$

- From the KKT dual complementarity condition, we will have $\alpha i > 0$ only for the training examples that have functional margin exactly equal to one

Optimal margin classifiers



- Consider the figure below, in which a maximum margin separating hyperplane is shown by the solid line.
- The points with the smallest margins are exactly the ones closest to the decision boundary
- Here, three points (one negative and two positive examples) that lie on the dashed lines parallel to the decision boundary.
- Thus, only three of the α_i 's will be non-zero at the optimal solution to our optimization problem.
- These three points are called the **support vectors** in this problem.
- The fact that the number of support vectors can be much smaller than the size the training set will be useful later

What is Hinge Loss?

- Hinge loss is used in SVM to ensure correct classification.
  - It penalizes misclassified points and those within the margin.

- Loss function:

$$L(y, f(x)) = \max(0, 1 - y * f(x))$$

  - If correctly classified and outside margin $\longrightarrow$ loss = 0.
  - Otherwise, the loss increases linearly.

Mathematical Formulation of SVM

- Decision function: $f(x) = w^T x + b$
- Hinge loss function: $L(y, f(x)) = \max(0, 1 - y f(x))$
- Total objective function:

  $$J(w, b) = (1/N) \sum \max(0, 1 - y_i (w^T x_i + b)) + (\lambda/2) ||w||^2$$

  - The first term is the hinge loss.
  - The second term is the regularization to prevent overfitting.

Visualization of SVM and Hinge Loss

- SVM aims to maximize the margin between two classes.
- Support vectors define the optimal hyperplane.
- Points within the margin or misclassified contribute to the loss.
- The hinge loss increases when a point is incorrectly classified.

- SVM is a powerful classification algorithm.
- Hinge loss ensures correct classification and large margin.
- The objective function balances hinge loss and regularization.
- Useful in high-dimensional spaces and robust to overfitting.

Reference

- Machine Learning, Stanford University by Prof. Andrew NG (Coursera)
- Support Vector Machines (SVMs): A friendly introduction by Luis Serrano